SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1709**

CO1-AT2-Constraint-Based Problem Solving

Register Number	**192312620**
Name	**V.Madhulika**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 20%

QUESTIONS:3 **Total Marks:30**

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

   Constraints:

   i. D1 cannot work Night shift
   ii. D2 must work before D3 (Morning < Afternoon < Night)
   iii. Only one doctor per shift
   iv. D3 cannot work Morning
   v. All doctors must be assigned exactly one shift

   Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

   Constraints:

   i. Movement allowed: Up, Down, Left, Right
   ii. Cost of each move = 1
   iii. Cannot pass through obstacles
   iv. Use Manhattan Distance heuristic

   Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost**.**

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.
   The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

   Constraints:

   - The robot can move up, down, left, right

- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:
a) Analyze all the given constraints and explain how they affect the robot's decision-making process.
b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
c) Demonstrate how the robot applies AI concepts such as:
- Intelligent agents
- Rationality
- Environment type

d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

Code of Conduct:

I V.Madhulika(192312620) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

V.Madhulika

Signature of the student:

RUBRICS FOR CALCULATION:

Criterion	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Criteria	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**	**Max Marks**
Problem Understanding	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly	6
Constraint Analysis	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization	6
Solution Development	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints	7.5

Application of Concepts	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts	6
Justification	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided	4.5

Question 1: Hospital Doctor-Shift Scheduling (Backtracking Search / CSP)

Assign doctors D1, D2, D3 to shifts Morning, Afternoon, Night (one doctor per shift) satisfying all constraints.

CSP Formulation

Component	Definition
Variables	D1, D2, D3
Domain (each variable)	{Morning, Afternoon, Night}
Constraint 1	D1 ≠ Night
Constraint 2	D2 must be assigned an earlier shift than D3 (order: Morning < Afternoon < Night)
Constraint 3	All-different: no two doctors share the same shift (one doctor per shift)
Constraint 4	D3 ≠ Morning
Constraint 5	Every doctor gets exactly one shift (complete assignment)

Backtracking Search Trace

Variables are assigned in the order D1 → D2 → D3. Domain values are tried in the order Morning → Afternoon → Night. A branch is abandoned (backtrack) the moment a constraint check fails.

Assign D1 = Morning
  Check (D1 ≠ Night): Morning ≠ Night -> OK, consistent

 Assign D2 = Morning -> violates all-different (Morning taken by D1) -> SKIP
 Assign D2 = Afternoon
   No direct constraint on D2 alone -> OK, consistent

   Assign D3 = Morning -> violates (D3 ≠ Morning) -> BACKTRACK
   Assign D3 = Afternoon -> violates all-different (taken by D2) -> SKIP
   Assign D3 = Night
     Check (D3 ≠ Morning): Night ≠ Morning -> OK
     Check (D2 before D3): Afternoon(2) < Night(3) -> OK
     All constraints satisfied -> SOLUTION FOUND

>> D1 = Morning, D2 = Afternoon, D3 = Night

If the search is continued (to verify no earlier branch was missed), backtracking to D2 = Night is explored next:

Assign D2 = Night -> OK, consistent (no direct constraint yet)
Assign D3 = Morning -> violates (D3 ≠ Morning) -> BACKTRACK
Assign D3 = Afternoon
Check (D2 before D3): Night(3) < Afternoon(2)? FALSE -> violates constraint 2 -> BACKTRACK
Assign D3 = Night -> violates all-different (taken by D2) -> SKIP
No valid D3 for D2 = Night -> D2 branch exhausted -> BACKTRACK to D1

Continuing the search from D1 = Afternoon (next domain value) yields a second valid solution:

Assign D1 = Afternoon -> Check (D1 ≠ Night): OK
Assign D2 = Morning -> OK
Assign D3 = Night
Check (D3 ≠ Morning): OK
Check (D2 before D3): Morning(1) < Night(3) -> OK
All constraints satisfied -> SOLUTION FOUND
>> D1 = Afternoon, D2 = Morning, D3 = Night

Assign D1 = Night -> violates (D1 ≠ Night) immediately -> BACKTRACK (branch pruned, D2/D3 never assigned)

Final Valid Schedule

Doctor	Shift
D1	Morning
D2	Afternoon
D3	Night

This is the first solution returned by the backtracking search (search order D1→D2→D3, domain order Morning→Afternoon→Night). A second valid schedule also exists - D1 = Afternoon, D2 = Morning, D3 = Night - since the given constraints do not uniquely pin down D1 and D2, only D3's position (Night).

Question 2: Robot Path-Finding in a 5×5 Grid

The question asks for step-by-step node expansion with a priority queue driven by the Manhattan-distance heuristic. A plain FIFO-queue BFS does not use a heuristic or a priority queue, so this is solved as a priority-queue-based Best-First / A search (f(n) = g(n) + h(n)), which is the standard way this requirement is implemented*

Grid Setup

Rows and columns are numbered 0-4. S = Start = (0,0). G = Goal = (4,4). Obstacles (#) block cells (0,2), (1,2), (2,2).

Row \ Col	0	1	2	3	4
0	S	.	#	.	.
1	.	.	#	.	.
2	.	.	#	.	.
3	.	.	.	.	.
4	.	.	.	.	G

Heuristic used: h(n) = |row - 4| + |col - 4| (Manhattan distance to the goal). Move cost g = 1 per step. f(n) = g(n) + h(n).

Step-by-Step Node Expansion and Priority-Queue Updates

Step	Node expanded (g, h, f)	New nodes generated & pushed to priority queue
1	(0,0) g0 h8 f8	(0,1) g1 h7 f8 | (1,0) g1 h7 f8
2	(0,1) g1 h7 f8	(1,1) g2 h6 f8
3	(1,0) g1 h7 f8	(2,0) g2 h6 f8
4	(1,1) g2 h6 f8	(2,1) g3 h5 f8
5	(2,0) g2 h6 f8	(3,0) g3 h5 f8
6	(2,1) g3 h5 f8	(3,1) g4 h4 f8
7	(3,0) g3 h5 f8	(4,0) g4 h4 f8
8	(3,1) g4 h4 f8	(3,2) g5 h3 f8 | (4,1) g5 h3 f8
9	(3,2) g5 h3 f8	(3,3) g6 h2 f8 | (4,2) g6 h2 f8
10	(3,3) g6 h2 f8	(3,4) g7 h1 f8 | (4,3) g7 h1 f8
11	(3,4) g7 h1 f8	(4,4) g8 h0 f8 -> GOAL generated
12	(4,4) g8 h0 f8	GOAL popped from priority queue -> STOP

At every expansion, all frontier nodes share f = 8 (this grid's obstacle gap keeps every explored path on an optimal footing), so ties are broken by picking the node with the smaller h (closer to the goal) / earliest insertion, which is why the search proceeds smoothly along the boundary of the obstacle wall down to row 3 and then across to the goal. Nodes that would revisit an already-expanded cell, or reach a cell with no improvement in g, are discarded (not re-pushed).

Optimal Path and Cost

Reconstructing the path via parent pointers from the goal back to the start:

(0,0) → (0,1) → (1,1) → (2,1) → (3,1) → (3,2) → (3,3) → (3,4) → (4,4)

Total path length = 8 moves. Total path cost = 8 × 1 = 8.

This equals the Manhattan distance between S and G (|4-0|+|4-0| = 8), confirming the path is optimal - the obstacle wall in column 2 has a gap at row 3 that the heuristic-guided search finds without any wasted detour.

Question 3: Autonomous Rescue Robot (Partially Observable Environment)

a) Analysis of Constraints and Their Effect on Decision-Making

Constraint	Effect on the robot's decision-making
Movement: Up / Down / Left / Right	Defines the action set / branching factor (up to 4 successors per cell); the successor function only returns in-bounds, non-obstacle neighbours
Blocked cells (obstacles)	Shrinks the usable state space; every candidate move must pass a validity check before being added to the frontier
Limited sensing (only adjacent cells observable)	Makes the environment partially observable to the robot - it cannot compute a complete path in advance and must interleave sensing, planning, and acting (an online search problem)
Move cost = 1, risky-zone cost = +2 (total 3)	Creates a non-uniform-cost graph, so a plain unit-cost search (like BFS) cannot guarantee the cheapest route; the agent needs a cost-aware strategy (UCS-style)
Minimize total cost while staying safe	Turns the task into a constrained optimisation: the robot must trade off shorter/riskier routes against longer/safer ones rather than just minimising the number of steps
Dynamic knowledge update (online search)	Forces the robot to continuously revise its internal map as new cells are sensed and to be able to replan or backtrack when a previously unknown obstacle is discovered

b) Solution Approach: Online Cost-Based Search Agent

Because the map is not fully known in advance and must be discovered through limited sensing, an Online Search Agent built around Uniform-Cost-Search principles (an online/incremental UCS, similar in spirit to LRTA*) is used rather than a purely offline algorithm. The robot interleaves acting and planning:

1. Initialise: place the robot at S; its internal knowledge map is empty except for what its sensors currently detect (adjacent cells).
2. Sense: at every cell, check the up/down/left/right neighbours to detect obstacles or risky zones within sensing range.
3. Update knowledge: record the sensed cells (free / obstacle / risky) into the robot's internal partial map.
4. Estimate cost: assign edge cost 1 to normal moves and 3 (1 + 2 risk penalty) to moves into a risky zone.

5. Select next move: from the reachable, already-sensed neighbours, choose the one with the lowest cumulative cost g(n) (as in Uniform Cost Search); ties can be broken using a Manhattan-distance estimate to G to bias movement toward the goal.

6. Act: move the robot to the selected cell and add the move's cost to the running total.

7. Backtrack if needed: if the robot senses that its current path leads to a dead end (all neighbours blocked or already visited with no cheaper alternative), it backtracks to the last cell with an unexplored, lower-cost option - a capability plain offline search does not need but an online agent must have.

8. Repeat steps 2-7 until the robot reaches G, then report the total accumulated cost of the actual path travelled.

c) Demonstration of AI Concepts

- Intelligent agent: the robot continuously perceives its environment through limited sensing (percepts = status of adjacent cells), reasons about the best next move, and acts by moving - the classic perceive-decide-act cycle. PEAS: Performance = rescue survivor at minimum cost while staying safe; Environment = the disaster-affected building grid; Actuators = movement (Up/Down/Left/Right); Sensors = adjacent-cell detectors.
- Rationality: the robot is rational because, given only the information it has sensed so far, it always chooses the action that it expects will minimise the performance measure (total cost) while avoiding known hazards - rationality here does not require omniscience, only the best decision possible with current knowledge.
- Environment type: partially observable (sensing is limited to adjacent cells, not the whole grid); sequential (each move affects future options); dynamic in the sense that the robot's knowledge of the world changes over time even though the physical grid itself is fixed per episode; discrete (grid cells and moves); single-agent; largely deterministic (a move succeeds as expected), though risky zones represent an added hazard/cost rather than uncertainty in outcome.

d) Justification of the Chosen Approach

- Offline algorithms (plain BFS/DFS or a pre-computed A*) assume the entire map is known before search begins - that assumption is violated here because the robot can only sense adjacent cells, so an online search agent that plans and acts incrementally is the only architecture consistent with the partial-observability constraint.
- Because moves have non-uniform cost (1 for normal, 3 for risky), a cost-blind method like BFS would not necessarily find the minimum-cost path; using Uniform-Cost-Search principles inside the online agent guarantees the robot always expands/chooses the lowest-cost known option, directly satisfying the 'minimize total cost' requirement.
- The ability to dynamically update its internal map and backtrack when new obstacles are discovered directly satisfies the constraint that the robot must 'update its knowledge dynamically' - a capability that is core to online search agents but absent from static offline search.
- Overall, the online cost-based search agent is the most suitable choice because it simultaneously respects all three defining constraints of the problem: partial observability, non-uniform movement cost, and the need for dynamic, on-the-fly replanning.